



PROJECT PLAN

Car and Rental Equipment Management System

CREMS

Client: Carpenters Fiji Pte Ltd

CS400 Industry Experience Project | Semester 2, 2026

Project detail	Information
Group number	[INSERT GROUP NUMBER]
Project supervisor	Amit Kumar
Co-supervisor	Shanil Naidu
Version / date	Version 1.0 4 August 2026

Team member	Student ID
Kavish Chandra	[INSERT STUDENT ID]
[INSERT TEAM MEMBER 2]	[INSERT STUDENT ID]
[INSERT TEAM MEMBER 3]	[INSERT STUDENT ID]
[INSERT TEAM MEMBER 4]	[INSERT STUDENT ID]



Document control

Version	Date	Prepared by	Change
1.0	4 Aug 2026	CREMS project team	Initial project plan for review

Approval record

Role	Name	Approval / signature	Date
Project leader	[INSERT NAME]		
Client representative	[INSERT NAME]		
Academic supervisor	Amit Kumar		

Executive summary

Carpenters Fiji currently requires a centralized way to coordinate vehicle and equipment rentals across branches. CREMS will replace fragmented, manual and spreadsheet-based activities with a secure web application covering assets, customers, reservations, agreements, inspections, returns, maintenance, notifications and management reporting.

The team will deliver the project as a modular web application using React, ASP.NET Core and Microsoft SQL Server. Work will be completed iteratively over 14 weeks, with requirements validation, fortnightly demonstrations and client acceptance checkpoints. The minimum viable product prioritizes accurate availability, role-based access, a traceable rental lifecycle and reliable operational reporting.

Delivery recommendation: Protect the MVP boundary. Accounting/ERP integration, online payments, GPS tracking, mobile applications and external booking platforms remain out of scope unless approved through formal change control.

Project at a glance

Dimension	Plan
Duration	14 weeks: 3 August to 8 November 2026
Team	Four student developers with shared delivery responsibility
Method	Agile, two-week iterations, client review at each major milestone
Target users	Administrators, branch managers, rental officers and customers
Deployment	Responsive web frontend, REST API and relational database
Budget basis	Academic project; labour and existing hardware treated as in-kind contributions



1. Background and problem analysis

1.1 Client context

Carpenters Fiji manages vehicles and rental equipment that move through reservation, allocation, inspection, rental, return, maintenance and reporting activities. These activities involve multiple people and branches and require dependable, current information. Delays or inconsistencies can cause double bookings, unavailable assets being promised, incomplete condition evidence, missed maintenance and weak management visibility.

1.2 Problem statement

The core problem is the absence of a single, role-controlled source of truth for rental assets and transactions. Operational records that are distributed across paper, spreadsheets or isolated processes make it difficult to confirm availability, preserve a complete transaction history and produce timely reports. CREMS must centralize these records while remaining usable for frontline staff and manageable by a four-person project team.

1.3 Information requirements

- Asset identity, category, registration or serial details, branch, availability state and maintenance status.
- Customer identity and contact information, rental history, agreements and deposits.
- Booking dates, assigned assets, pricing references, extensions, returns and overdue status.
- Pre-rental and post-rental inspection results, damage, fuel, accessories and supporting notes.
- Role, branch and account status for each user, plus an audit trail of critical changes.
- Operational measures for utilization, revenue, overdue rentals, maintenance and customer history.

1.4 Constraints and assumptions

Type	Constraint or assumption	Planning response
Time	One academic semester and fixed assessment deadlines.	Prioritize a demonstrable MVP and use staged acceptance.
People	Four students working around other courses.	Clear ownership, peer review and weekly workload checks.
Requirements	Some business rules remain subject to client confirmation.	Maintain a requirements register and decision log.
Technology	Solution must be supportable on team laptops and a realistic server environment.	Use cross-platform .NET, React, Docker and SQL Server.
Privacy	Customer and identity data must not appear in repositories or demonstrations.	Use synthetic data, secrets management and least privilege.
Connectivity	Development and client review may occur across different networks.	Local environments plus portable demonstrations and backups.



2. Objectives and success measures

2.1 Project objectives

- Deliver one centralized, responsive web application for vehicle and equipment rentals across branches.
- Prevent conflicting reservations through authoritative availability and date validation.
- Support the rental lifecycle from booking and agreement through allocation, inspection, return and closure.
- Give each user only the functions and records permitted by role and branch.
- Improve traceability through transaction history, inspection evidence and audit events.
- Provide dashboards and reports that support timely operational decisions.

2.2 Measurable acceptance criteria

ID	Acceptance measure	Evidence
AC-01	Authorized users can sign in and receive the correct navigation and API permissions for all four roles.	Role-based API and UI test results
AC-02	The system rejects an overlapping confirmed booking for the same asset and date range.	Automated test and demonstration
AC-03	Staff can complete booking, allocation, inspection, return and closure using test data.	End-to-end acceptance scenario
AC-04	Dashboard totals reconcile with underlying test transactions.	Report reconciliation checklist
AC-05	Critical create/update/status actions record who acted and when.	Audit-log inspection
AC-06	Core screens work at desktop and mobile widths and meet agreed usability checks.	Responsive test record
AC-07	No critical or high-severity security defect remains open at handover.	Security checklist and issue register

3. Scope

3.1 In scope - MVP

- Authentication and role-based access for Administrator, Branch Manager, Rental Officer and Customer.
- Branch, vehicle and equipment master records, including status and identification code.
- Customer registration and profile management.
- Booking, availability checking, allocation, agreement, extension, return and closure.
- Inspection checklists, fuel/accessory records, damage reports and maintenance requests.
- Preventive maintenance schedules and reminders.
- Dashboards and agreed rental, utilization, maintenance, revenue, overdue and customer reports.
- Email notifications for booking, overdue rental and scheduled maintenance events.
- Audit logs for critical transactions and asset movements.



3.2 Out of scope

- Accounting or ERP integration.
- Online payment gateway.
- Native mobile application.
- GPS vehicle tracking.
- Third-party insurance integration.
- Driver scheduling and payroll.
- Loyalty or rewards programmes.
- External booking-platform integration.

3.3 Future options

The architecture will preserve REST API boundaries and stable identifiers so approved future work can add payment, ERP, telematics or mobile clients without redesigning the MVP. These are design considerations, not current deliverables.

4. Solution approach and feasibility

4.1 Selected solution

Layer	Technology	Reason
Frontend	React 19, TypeScript, Vite, Material UI	Responsive component model, fast development and strong type checking.
Backend	ASP.NET Core 10 Web API	Enterprise authorization, validation, performance and maintainability.
Data	Microsoft SQL Server with Entity Framework Core	Relational integrity, migrations and natural alignment with the client brief.
Identity	ASP.NET Core Identity with cookie authentication	Secure account management and server-enforced roles.
Delivery	GitHub, Docker Compose, Postman and VS Code	Repeatable environments, reviewable changes and API verification.

4.2 Alternatives considered

Alternative	Strength	Reason not selected
Laravel + React	Rapid CRUD development and approachable PHP ecosystem.	Would require rewriting completed identity/API work and is less aligned with SQL Server and the approved brief.
Java Spring + React	Mature enterprise platform and strong typing.	Higher setup overhead for the current team with no material project advantage.
No-code/SaaS	Fast prototype creation.	Insufficient control over rental rules, auditability, extensibility and academic software-development outcomes.



4.3 Feasibility assessment

Dimension	Assessment	Conclusion
Technical	The stack is cross-platform and the team has already established authentication, database migrations, Docker and a React shell.	Feasible with disciplined testing.
Operational	Browser-based workflows reduce installation burden; role-specific screens support staff responsibilities.	Feasible subject to client validation.
Schedule	Four people over 14 weeks can deliver the MVP if integrations remain out of scope.	Feasible with strict change control.
Economic	Open-source development tools and existing student hardware keep incremental expenditure low.	Feasible within academic constraints.
Ethical/legal	Synthetic test data, least privilege, secure secrets and documented consent/privacy controls reduce exposure.	Feasible with ongoing review.

5. Delivery method and work breakdown

5.1 Agile operating model

The team will work in two-week iterations. Each iteration begins with backlog refinement and task ownership, uses pull requests and peer review during implementation, and ends with an integrated demonstration. Client decisions are recorded in the requirements register. A feature is complete only when code, validation, tests and documentation are integrated.

5.2 Work packages

WP	Work package	Primary outputs
WP1	Discovery and planning	Validated requirements, scope, project plan, team charter, initial backlog
WP2	Architecture and environments	Repository, Docker database, CI-ready builds, data model, API conventions
WP3	Identity and administration	Authentication, roles, branch-aware accounts, user administration
WP4	Assets and customers	Asset/equipment catalogue, customer records, identification codes
WP5	Booking and availability	Reservation workflow, conflict rules, allocation and agreement
WP6	Return, inspection and maintenance	Condition evidence, damage, fuel, maintenance scheduling
WP7	Dashboard, reports and notifications	Operational measures, exports and email events
WP8	Quality, deployment and handover	Security/performance tests, UAT, deployment guide, training and final report

5.3 Definition of done

- Acceptance criteria are clear and linked to a requirement.



- Implementation builds successfully and follows agreed conventions.
- At least one other member reviews the change.
- Automated or repeatable manual tests pass.
- Authorization is enforced by the API, not only hidden in the interface.
- Documentation and demonstration data are updated.
- The product owner/client accepts the outcome or records follow-up work.

6. Schedule and milestones

Dates are proposed for Semester 2, 2026 and should be confirmed with the client and academic supervisor. Work packages overlap deliberately so analysis, implementation and validation progress continuously.

Weeks / dates	Focus	Owner(s)	Exit deliverable
W1-2 3-16 Aug	Discovery, domain research, plan, charter and requirements baseline	All; Leader coordinates	Approved plan and prioritized MVP backlog
W3 17-23 Aug	Architecture, database refinement and environment standardization	Backend + DevOps leads	Repeatable full-stack setup
W4 24-30 Aug	Identity, roles, branches and user administration	Backend + Frontend leads	Role-based access demonstration
W5-6 31 Aug-13 Sep	Asset/equipment and customer management	Pair A / Pair B	Asset and customer modules
W7-8 14-27 Sep	Bookings, availability, allocation and agreements	All, feature pairs	End-to-end booking demonstration
W9 28 Sep-4 Oct	Returns, extensions and closure	Rental workflow pair	Completed rental lifecycle
W10 5-11 Oct	Inspections, damage and maintenance	Asset workflow pair	Inspection and maintenance modules
W11 12-18 Oct	Dashboard, reports, QR/barcodes and notifications	Frontend/reporting pair	Management information demonstration
W12 19-25 Oct	Integration, accessibility, security and performance testing	All; QA lead coordinates	Release candidate 1
W13 26 Oct-1 Nov	Client UAT, defect correction and training material	All; Client liaison coordinates	UAT acceptance record
W14 2-8 Nov	Deployment rehearsal, final documentation and handover	All	Final release and handover package

6.1 Milestones

Milestone	Target	Acceptance condition
M1 - Plan baseline	16 Aug	Project plan, charter, scope, risks and backlog reviewed.
M2 - Technical foundation	30 Aug	All members can run frontend, API and database; role demonstration passes.



Milestone	Target	Acceptance condition
M3 - Core records	13 Sep	Assets and customers can be maintained with validation.
M4 - Rental workflow	4 Oct	Booking-to-return scenario succeeds without conflicting allocation.
M5 - Feature complete	18 Oct	All MVP modules integrated; only defects/documentation remain.
M6 - UAT complete	1 Nov	Agreed client scenarios pass or exceptions are formally recorded.
M7 - Handover	8 Nov	Release, source, database scripts, guides and final report delivered.

7. Resources and responsibilities

7.1 Human resources

Project role	Assigned person	Main responsibilities	Planned effort
Project leader / client liaison	[INSERT NAME]	Plan, client meetings, decisions, integration and submission control.	6 h/week
Backend and data lead	[INSERT NAME]	Domain model, API, database, authorization and backend tests.	6 h/week
Frontend and UX lead	[INSERT NAME]	Responsive UI, accessibility, workflow integration and UI tests.	6 h/week
Quality and DevOps lead	[INSERT NAME]	Test planning, Docker, build checks, release evidence and documentation.	6 h/week

Roles establish accountability but do not create silos. Members will pair on high-risk features, review each other's pull requests and rotate meeting-note and demonstration duties.

7.2 Technical and information resources

- Four development laptops with Git, .NET 10 SDK, Node.js 22+, Docker Desktop, VS Code and Postman.
- GitHub repository for source, pull requests, issues and protected review workflow.
- SQL Server 2022 container for consistent local databases.
- Client representatives for workflow validation, sample forms and acceptance decisions.
- Academic supervisor for assessment guidance and escalation.
- Synthetic test dataset; no production customer data stored in student environments.

7.3 RACI summary

Activity	Leader	Backend	Frontend	QA/DevOps	Client
Requirements / scope	A	C	C	C	R/C
Architecture / data	C	A/R	C	C	I
User experience	C	C	A/R	C	C
Testing / release	C	R	R	A/R	C



Activity	Leader	Backend	Frontend	QA/DevOps	Client
Acceptance / change approval	R	C	C	C	A

Legend: A = accountable, R = responsible, C = consulted, I = informed.

8. Budget

This is an academic project. Student labour, personal laptops and existing internet access are in-kind contributions and are shown for transparency but are not charged to Carpenters Fiji. Any paid service requires prior written approval.

Item	Basis	Cash cost (FJD)	Treatment / justification
Student development effort	4 people x 6 h/week x 14 weeks = 336 hours	0	In-kind academic contribution.
Development laptops	Existing personal equipment	0	No new purchase required.
Software/tooling	.NET, React, VS Code, GitHub, Postman and Docker student/personal use	0	Open-source or existing free access.
Local database	SQL Server Developer container	0	Development/test use only.
Cloud demonstration hosting	Optional short-lived test deployment	120	Only if client requires remote access.
Email/domain allowance	Optional test mailbox/domain	50	Supports realistic notification testing.
Contingency	Approx. 20% of possible cash expenditure	35	Covers minor approved service costs.
Total maximum cash budget		205	Spend only with approval; expected cost may remain zero.

Budget control: The project leader records approved expenditure and evidence. Scope will not be expanded merely because contingency remains available.

9. Risk management

Risks are reviewed weekly. Probability (P) and impact (I) use a 1-5 scale; exposure is P x I. Scores 15-25 are High, 8-14 Medium and 1-7 Low.

ID	Risk	P	I	Rating	Response / owner
R1	Requirements remain unclear or change late.	4	5	20 H	Fortnightly validation, decision log and change control / Leader
R2	Scope exceeds semester capacity.	4	5	20 H	Protect MVP, prioritize backlog and defer integrations / All
R3	Double-booking or status rules are incorrect.	3	5	15 H	Domain rules, transactions and overlap tests / Backend
R4	One member becomes unavailable.	3	4	12 M	Shared documentation, pairing and small reviewed changes / Leader



ID	Risk	P	I	Rating	Response / owner
R5	Integration occurs too late.	3	4	12 M	Continuous main-branch integration and iteration demos / DevOps
R6	Sensitive data or credentials are exposed.	2	5	10 M	Synthetic data, user secrets, review and secret scanning / All
R7	Authorization allows excess access.	3	5	15 H	Server policies plus role-based negative tests / Backend + QA
R8	Environment differences block a team member.	3	3	9 M	Docker, setup guide and verified Mac/Windows steps / DevOps
R9	Client availability delays acceptance.	3	4	12 M	Book reviews early and submit focused decision requests / Liaison
R10	Data loss or repository damage.	2	5	10 M	GitHub remote, protected branches and database backup rehearsal / DevOps

9.1 Escalation thresholds

- A High risk is discussed immediately with the project leader and given a dated mitigation task.
- A decision that changes scope, delivery date, privacy posture or architecture is escalated to the client/supervisor.
- A blocked task lasting more than two working days is raised at the next team contact, not held until the iteration review.

10. Quality, security and professional practice

10.1 Quality plan

Control	Minimum evidence
Build quality	Backend build, frontend TypeScript build and lint checks pass.
Code review	At least one peer approval for functional changes.
API testing	Positive, validation, unauthenticated and unauthorized cases in Postman/automated tests.
Data quality	Required fields, relationships, uniqueness and date constraints verified.
Usability	Agreed workflows tested at desktop and mobile widths.
Release	UAT checklist, known-issues list and deployment rehearsal completed.

10.2 Security and privacy

- Use least-privilege roles and branch boundaries; enforce access in API policies and queries.
- Store passwords through ASP.NET Identity hashing; never log or display passwords.
- Keep connection strings and bootstrap credentials in local secret stores, not Git.
- Use HTTPS in deployed environments, secure cookies and trusted-origin configuration.



- Collect only information needed for rental operations and define retention/deletion expectations with the client.
- Use synthetic customer and transaction data for development, demonstrations and assessment evidence.
- Record critical user, rental and asset status changes in an auditable form.

10.3 Ethical and professional conduct

The team will protect client confidentiality, accurately report progress and limitations, respect software licences, acknowledge sources and avoid presenting generated or unverified material as client-approved fact. Safety-sensitive records such as vehicle condition and maintenance must not be altered without traceability. Team decisions will consider the interests of customers, staff, the client and the wider public.

11. Communication, governance and change control

Forum	Frequency	Participants	Output
Team stand-up	Twice weekly	Four students	Progress, next actions and blockers
Planning/review	Every two weeks	Team; client when available	Iteration goal, demonstration and accepted feedback
Client checkpoint	Fortnightly or as agreed	Liaison, relevant members, client	Decisions, clarified rules and acceptance notes
Supervisor update	As scheduled	Leader/team and supervisor	Academic guidance and escalations
Pull-request review	For each change	Author plus reviewer	Technical review and evidence
Risk/scope review	Weekly	Whole team	Updated registers and mitigation owners

11.1 Change process

1. Record the requested change, reason and requesting stakeholder.
2. Estimate value, effort, schedule effect, risk and impact on existing acceptance criteria.
3. Recommend accept, defer, substitute or reject.
4. Obtain approval from the project leader and client for material scope changes.
5. Update scope, backlog, schedule, risk register and decision log before implementation.

11.2 Status reporting

The weekly status summary will show completed work, next-week commitments, milestone confidence, top risks, decisions required and workload concerns. Status will be evidence-based using integrated software, issue history and acceptance results.



12. Deliverables and handover

Deliverable	Contents	Acceptance owner
D1 Project governance pack	Project plan, charter, requirements register, risks, decisions and meeting records.	Supervisor / client
D2 Source and database	Reviewed source, migrations, seed guidance and version history.	Technical representative
D3 CREMS MVP	Responsive application covering accepted in-scope workflows.	Client representative
D4 Test and acceptance pack	Test cases/results, security checks, UAT record and known issues.	Client + QA lead
D5 Deployment and operations pack	Environment settings, deployment, backup/restore and support notes.	Technical representative
D6 User documentation	Role-based quick guides and administrator instructions.	Client representative
D7 Final academic report/demo	Project outcomes, evaluation, reflection and demonstration.	Academic supervisor

12.1 Handover checklist

- Final release is tagged and reproducible from documented prerequisites.
- Production secrets are supplied through an approved private channel.
- Database migration and backup/restore steps are demonstrated.
- Administrator and representative operational users complete training.
- Known limitations and deferred items are documented.
- Client acceptance, outstanding actions and support ownership are recorded.

13. Monitoring and control

Measure	Target	Review
Milestone completion	All seven milestones accepted or formally re-baselined.	Weekly
Iteration predictability	At least 80% of committed priority items completed.	Fortnightly
Build health	Main branch builds successfully.	Every integration
Defect position	Zero open critical/high defects at handover.	Weekly from W10
Review coverage	Functional changes receive peer review.	Every pull request
Client decisions	Open decision requests receive an owner and due date.	Weekly
Team health	Workload is visible and no critical knowledge is held by one member.	Weekly



13.1 Project closure criteria

The project closes when accepted MVP scenarios pass, the agreed release and documents are handed over, critical/high defects are closed, known limitations are acknowledged, and the team completes academic submission and retrospective obligations.

14. References

Carpenters Fiji Pte Ltd. (2026). Car and Rental Equipment Management System project description and scope. Client-provided project brief.

The University of the South Pacific. (2026). CS400 Industry Experience Project, Assignment 1 - Project Plan specification and marking rubric.

Microsoft. (2026). ASP.NET Core documentation. <https://learn.microsoft.com/aspnet/core/> (accessed 4 August 2026).

Microsoft. (2026). Entity Framework Core documentation. <https://learn.microsoft.com/ef/core/> (accessed 4 August 2026).

React Team. (2026). React documentation. <https://react.dev/> (accessed 4 August 2026).

OWASP Foundation. (2026). Application Security Verification Standard. <https://owasp.org/www-project-application-security-verification-standard/> (accessed 4 August 2026).

15. Declaration of AI-assisted work

Generative AI assistance was used to help structure this project-plan draft, improve wording and propose planning artefacts such as the initial schedule, risk register and acceptance measures. The student team is responsible for verifying every statement, correcting assumptions, replacing placeholders, aligning the plan with client decisions and disclosing this use in accordance with the assignment specification. AI-generated content is not treated as client approval, evidence or a substitute for the team's own analysis.



Appendix A - Team charter

A.1 Purpose

We will work as one accountable team to deliver a secure, usable and evidence-based CREMS MVP for Carpenters Fiji while meeting CS400 academic and professional expectations.

A.2 Team commitments

- Attend scheduled meetings or notify the team early when unavailable.
- Complete agreed work by the stated date and raise blockers within two working days.
- Use Git branches, focused commits and pull requests; do not overwrite another member's work.
- Review peers constructively and discuss the work rather than the person.
- Protect client information and use synthetic data in development.
- Record material decisions, requirements and changes.
- Share knowledge through pairing, demonstrations and documentation.
- Report progress honestly and seek help before a deadline is endangered.

A.3 Decision-making and conflict resolution

1. Seek consensus using evidence, acceptance criteria and client value.
2. If consensus is not reached promptly, the project leader decides routine delivery matters after hearing each view.
3. Scope, policy or client-impacting decisions are referred to the client/supervisor.
4. Address interpersonal concerns privately and respectfully first; document agreed actions.
5. Escalate unresolved conduct or workload issues to the academic supervisor.

A.4 Working agreement

Area	Agreement
Primary channels	[INSERT TEAM CHAT] for coordination; GitHub for work/decisions; email for formal client communication.
Core response time	Acknowledge team messages within one working day.
Meetings	Twice-weekly team contact and fortnightly planning/review.
Code ownership	Collective ownership; authors obtain peer review before integration.
File naming	Clear, stable names; final submission controlled by project leader.
Definition of ready	Requirement has user value, acceptance criteria, dependencies and an owner.
Definition of done	Meets Section 5.3 and is integrated, tested, reviewed and documented.



A.5 Expected contribution and accountability

Each member plans approximately six project hours per week, adjusted transparently around assessment peaks. The team reviews workload weekly. Missed commitments are re-planned with an owner and date; repeated issues are discussed with the project leader and, if unresolved, the supervisor.

A.6 Charter acceptance

Team member	Student ID	Signature	Date
Kavish Chandra	[INSERT ID]		
[INSERT TEAM MEMBER 2]	[INSERT ID]		
[INSERT TEAM MEMBER 3]	[INSERT ID]		
[INSERT TEAM MEMBER 4]	[INSERT ID]		